Abstract

Comparison of Kubernetes-based Autoscaling on Proxmox VE

Nishant Singh Chauhan

Problem

The design of Kubernetes cluster Autoscaler is geared towards public clouds because a plugin from the provider converts the decision to scale up into an API request for a node pool. For the privately run Proxmox VE cloud, there is no such plugin available, which means that the integration layer must be developed manually. However, two other factors should be considered: the distribution used for Kubernetes cluster (kubeadm or lightweight K3s) and the underlying virtualization technology (isolated QEMU VM or LXC container).

Approach

The hypothesis performs a 2x2 controlled experiment using the two distributions in combination with the two substrates, creating four clusters (A: Kubeadm + VM, B: Kubeadm + LXC, C: K3s + VM, D: K3s + LXC). All four clusters are identically provisioned via OpenTofu and bpg/Proxmox, so any observed differences will be due to either distribution or substrate. A custom Python watcher (pending_watcher.py) identifies unschedulable pods and applies tofu, restricted via a Hypervisor Core Exhaustion Index (CEI) to prevent overloading of the host. HPA scales at the pod level, but the experiment concerns itself with cluster-level (node) scaling.

Key Results: Time-to-Scale (first pending pod to node Ready)

Configuration	Distribution + Substrate	Mean TTS	Interpretation
Config-D	K3s + LXC	56.9 s	Fastest baseline
Config-C	K3s + VM	79.2 s	“VM substrate tax” ≈ +39%
Config-A	kubeadm + VM	125.5 s	“Distribution tax” ≈ +58%; ~120% slower than D
Config-B	kubeadm + LXC	—	Architecturally infeasible

Config-B fails due to a lack of privileges for the LXC to run a CRI-based, multiple containers-based control plane, and CNI plugins by default require NET_ADMIN/NET_RAW capabilities which cannot be used in the host network namespace (sockets EPERM). The micro-phase analysis reveals that the VM tax occurs during clone/boot phases while the distribution tax is in the Kubernetes join/ready phase; K3s also uses about half the idle resources as compared to Kubeadm. This work disentangles the Time-to-Scale from the Time-to-Recovery (node is ready before latency recovers as the scheduler will not rebalance pods) and achieves zero dropped connection using MetalLB Layer 2 (ARP) failover, since the intended FreeBSD/BGP solution was not available.

Conclusion

These results make up a design space rather than an outright winner. The distribution tax (about 46 s) is by far the most concrete handle to grab, preferring K3s to Kubeadm is a pure software speed and density win, at the expense of some degree of upstream conformance. The VM substrate tax (about 22 s) is a conscious sacrifice of kernel isolation, appropriate for regulated or multi-tenant workloads and optional for trusted ones. More scalability means that the warm operator buffer can be smaller, resulting in cost savings.